

Informe de Programación Paralela: Balanceo de Carga y Sincronización

Ciencias de la Computación

25 de mayo de 2026

1. Ejercicio 3: Aproximación de la Función Zeta de Riemann

El objetivo de este ejercicio fue analizar e implementar la paralelización de un bucle con complejidad temporal $O(k^2)$ mediante el uso de hilos estándar de C++11 (`std::thread`).

1.1. Análisis de Dependencias y Variables Compartidas

- **Dependencias de Datos:** No existen dependencias de datos entre las iteraciones del bucle externo (*loop-carried dependencies*). Cada elemento $X[k]$ se calcula de forma independiente, clasificando el problema como *Embarrassingly Parallel*.
- **Variables Compartidas:** El vector de resultados X es compartido por todos los hilos. No obstante, dado que cada hilo escribe en índices estrictamente disjuntos determinados por el patrón de distribución, no se generan condiciones de carrera (*data races*), haciendo innecesario el uso de mecanismos de sincronización costosos.

1.2. Patrones de Distribución de Carga

Se evaluaron tres patrones de diseño fundamentales para la asignación de trabajo:

1. **Parallel Block (Bloques):** Divide el dominio N en subrangos continuos uniformes.
2. **Parallel Cyclic (Cíclico):** Distribuye las iteraciones de manera intercalada (Round Robin) de uno en uno.
3. **Parallel Block Cyclic (Bloque-Cíclico):** Variante híbrida que distribuye bloques de tamaño fijo (BS) de forma cíclica.

1.3. Evaluación Experimental y Resultados

La asimetría del cálculo (las iteraciones con k alto requieren drásticamente más tiempo que las iniciales) genera un severo desbalanceo de carga en la distribución por bloques. A continuación se presentan los tiempos medidos en milisegundos:

Cuadro 1: Resultados de las Pruebas de Rendimiento (en ms)

Configuración	Hilos	Size N	Cíclico	Bloques	Bloque-Cíclico
Prueba 1	4	2000	12,227	24,442	14,603 (BS=50)
Prueba 2	8	4000	56,291	134,588	71,269 (BS=100)

Cuadro 2: Prueba 3: Impacto del Tamaño de Bloque en Bloque-Cíclico ($N = 3500$, 8 Hilos)

Métrica / Variación	Cíclico Puro	Bloques Puros	Bloque-Cíclico	Tamaño Bloque (BS)
Prueba 3.1	39,769 ms	87,735 ms	41,360 ms	BS = 1
Prueba 3.2	39,928 ms	88,603 ms	84,923 ms	BS = 437
Prueba 3.3	39,671 ms	85,811 ms	43,027 ms	BS = 50

Conclusión del experimento: El patrón Cíclico minimiza los tiempos al garantizar que el "sufrimiento computacional" de los valores de k más altos se reparta equitativamente. La prueba 3.1 demuestra que con $BS = 1$ el comportamiento es idéntico al Cíclico, mientras que la prueba 3.2 con $BS = N/Hilos$ degenera en el patrón por Bloques. Al tratarse de un problema limitado por procesamiento (*CPU-bound*), el balanceo puro supera a la optimización de memoria caché.

2. Ejercicio 4: Sincronización con Variables de Condición

Se diseñó un mecanismo de unión (*custom join*) manual para detener el hilo principal hasta la completa finalización del hilo hijo utilizando variables de condición.

2.1. Implementación de Bloqueos (Scoped vs. Explicit)

Se contrastaron dos enfoques de exclusión mutua para la actualización de la variable de estado `done`:

- **Explicit Locking** (`m.lock()` / `m.unlock()`): Requiere la liberación manual del recurso. Presenta vulnerabilidades ante posibles excepciones en el flujo que impidan llegar al `unlock()`, provocando interbloqueos (*deadlocks*).
- **Scoped Locking** (`std::lock_guard`): Utiliza el patrón RAII. Garantiza la liberación segura y automática del mutex al salir del bloque de alcance (*scope*) delimitado por llaves.

Ambas implementaciones arrojaron el orden de salida esperado de manera consistente:

```
1 child
2 parent
```

3. Ejercicio 5: Sincronización One-Shot usando Futures y Promises

Para escenarios donde la comunicación ocurre una única vez (sincronización por ocurrencia de evento único o *One-Shot*), las variables de condición introducen una complejidad innecesaria.

A través de la sección 4.5 del texto guía, se reestructuró el programa utilizando `std::promise` y `std::future`. Este patrón elimina por completo las variables globales de control, los mutexes explícitos y la lógica de bucles condicionales del lado del receptor. El hilo secundario invoca `p.set_value()` (disparo único) desbloqueando limpiamente la barrera de espera del hilo principal configurada mediante `f.wait()`.